BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 255 · RISC and CISC

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Why can instruction fetch and data access contend in Von Neumann architecture?

Q2. What is a pipeline hazard?

Q3. What is an opcode?

Q4. Array [2,4,6,8,10,12,14,16,18], target 16. Give midpoint indexes inspected with low=0 high=8 and integer midpoint.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

RISC — Reduced Instruction Set Computer design approach.

CISC — Complex Instruction Set Computer design approach.

load/store — Access memory through dedicated instructions.

code density — Amount of program function per unit of machine-code storage.

opcode — Operation part of an instruction.

New learning

RISC approach — Typically favours a smaller set of comparatively simple, regular instructions, often using explicit load/store operations.

CISC approach — Can offer complex instructions that express several low-level operations in one instruction, potentially improving code density.

Avoid oversimplifying — Fewer instructions in assembly do not guarantee fewer clock cycles. Actual performance depends on pipelining, decoding, caching and processor design.

Worked example

Explain why a load/store architecture needs separate load and add instructions.

1. Predict first — Values are brought from memory into registers before the arithmetic instruction.

2. Explain and check — A regular load/store instruction set simplifies certain datapaths but may need more assembly instructions.

We Do · Practise with guidance

Two instruction sets compute and store A+B. One offers a single memory-add; the other requires four steps. Does one automatically run faster?

```
1 complex instruction vs 4 simpler instructions
Time also depends on cycles/decoding/cache.
```

Predict first

The complex ISA version uses fewer listed instructions but could require more work per instruction.

Explain and check

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Evaluate clock cycles, decode overhead, code size, memory behaviour and hardware rather than choosing by instruction count alone.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

State two characteristics of RISC and two of CISC.

Question 2

Compare one CISC-style memory-add instruction against a RISC-style LOAD, LOAD, ADD, STORE sequence for the same task.

Question 3

Evaluate the claim 'RISC is always faster because its instructions are simpler'. Include code density, decoding and pipelining.

Exit Ticket · Answer before checking

A program written using fewer assembly instructions always executes faster.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. They share a memory pathway or resource.

Q2. An event such as a dependency, branch or memory stall disrupting instruction overlap.

Q3. The operation part of a machine or assembly instruction.

Q4. mid4 value10 → low5 high8, mid6 value14 → low7 high8, mid7 value16 → found. Indexes 4,6,7.

You Do pages · Independent questions

Q1. [4] RISC typically simpler/regular instructions [1] and load/store designs [1]; CISC permits richer complex instructions [1] and often variable instruction lengths or complex addressing [1].

Q2. [6] The compound instruction can express more work in one opcode [2]; RISC separates memory access and arithmetic [2]; number of instructions is not the same as measured execution time [2].

Q3. [6] Simpler regular instructions may support efficient decoding [1] and pipelining [1]; more instructions may increase code size [1] and fetch demand [1]; complex instructions can improve code density [1]; real performance depends on implementation/workload [1].

Exit Ticket · Model answer

False. Instructions can differ in work and execution cost.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first.

GREEN PEN Then: tick correct answers; rewrite mistakes.

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check my understanding